

改进后软件过程定义文档

项目: yolink-front (悦联岛校园社交平台前端)

版本: v2.0 (AI 增强版)

作者: 殷浩杨 (2022141461109)

日期: 2026年6月

课程: 软件过程与管理, 四川大学软件学院, 2026年春

基于标准: ISO/IEC 12207:2017 + AI 过程扩展

前一版本: v1.0 (软件过程剪裁与定义, 2026年3月)

一、过程概述与目标

1.1 文档定位

本文档是 yolink-front 项目的完整软件过程定义, 在 v1.0 版本 (基于 ISO/IEC 12207:2017 的纯人工过程定义) 基础上系统性融入 AI 技术, 形成人机协同的智能化软件过程体系. 本文档替代 v1.0 版本, 作为项目后续开发活动的过程依据.

1.2 项目背景

yolink-front (悦联岛) 是面向在校学生群体的校园社交平台前端应用, 采用 `React 19` + `TypeScript` + `TanStack Router` + `TanStack Query` 技术栈, 移动优先响应式设计. 关键工程约束: 独立前端仓库开发与维护, 通过约定的 REST API 契约与后端通信, 部署流程与后端相互独立.

1.3 过程改进目标

- AI 增效: 将 AI 技术融入软件生命周期各阶段, 实现重复性活动的效率倍增
- 质量前移: 通过 AI 驱动的代码审查与测试生成, 将缺陷发现时机从测试阶段前移至编码阶段
- 人机协同: 建立 AI 生成与检测、人工决策与审查的职责边界
- 度量驱动: 建立含 AI 贡献度指标的过程度量体系

1.4 过程能力目标

依据 ISO/IEC 33001:2015 的过程能力等级模型:

过程类别	目标能力等级	说明
主要过程 (含 AI 辅助)	Level 2 (已管理)	过程活动有计划、有监控、有调整
支持过程 (含 AI 辅助)	Level 2 (已管理)	支持过程与主要过程协同管理
AI 能力管理过程 (新增)	Level 1 (已执行)	新型过程, 先确保真实执行再追求管理成熟度
组织过程	Level 1 (已执行)	维持在实用主义水平

二、过程框架总览

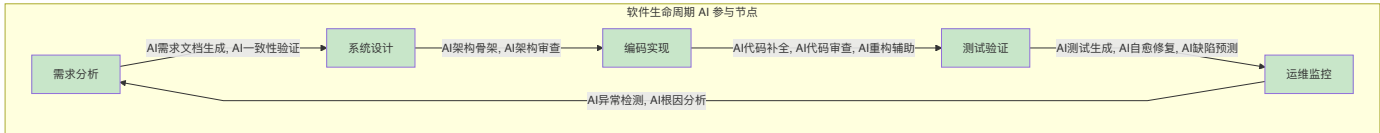
2.1 过程体系结构

本项目软件过程体系采用三层结构——主要过程、支持过程与组织过程——符合 ISO/IEC 12207:2017 顶层分类, 并新增 AI 增强层作为横切关注点嵌入各层过程.



Syntax error in text
mermaid version 11.9.0

2.2 AI 参与节点全景



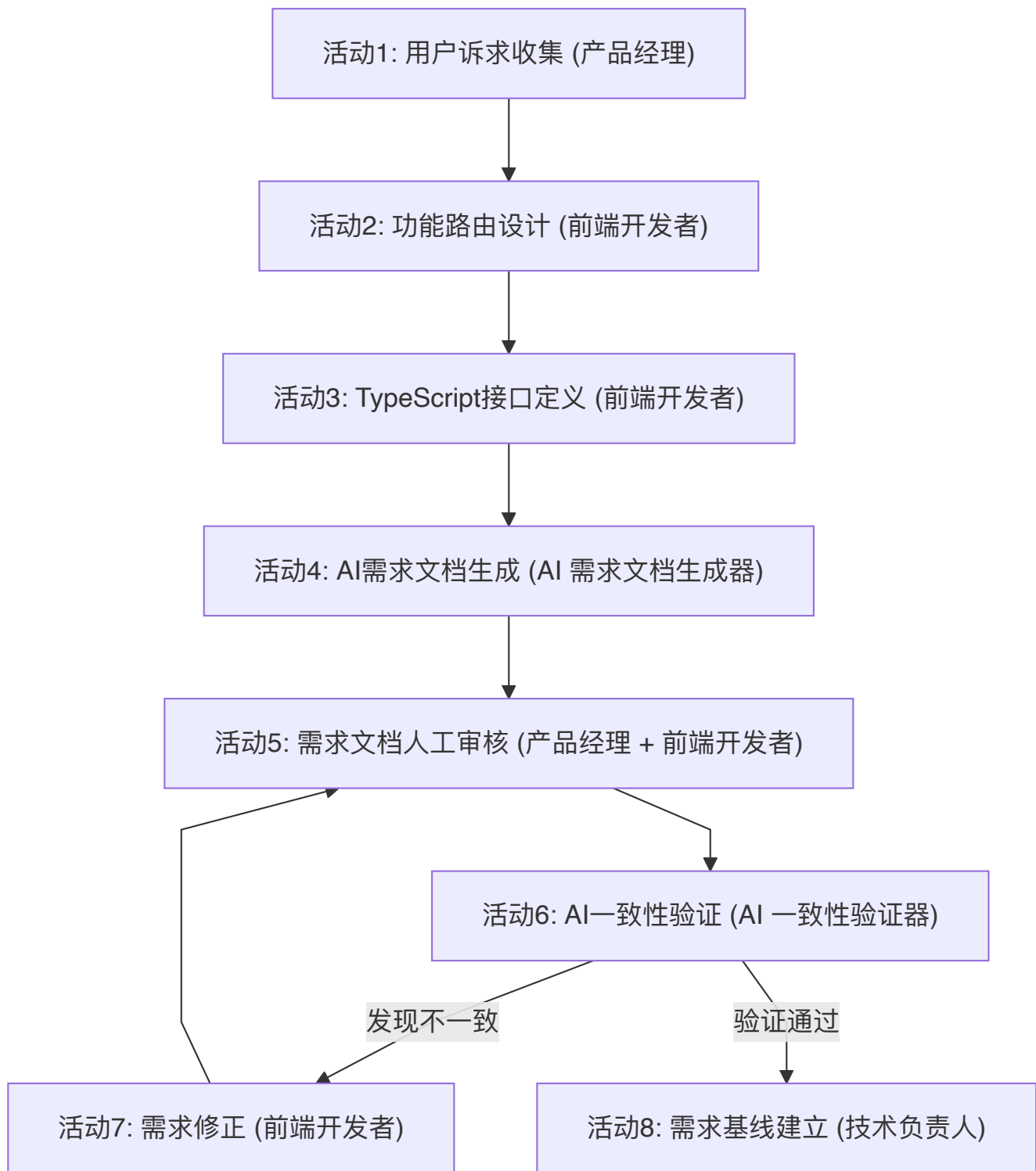
三、各阶段活动详细定义

3.1 需求分析过程 (主要过程)

3.1.1 过程概述

需求分析过程将用户诉求转化为可执行的功能规格. yolink-front 中需求分析活动以功能路由设计为核心产出物, TypeScript 类型定义为数据契约. v2.0 引入 AI 辅助——从路由与类型定义生成面向非技术利益相关方的用户故事文档, 并自动验证路由参数与类型字段的一致性.

3.1.2 活动流程



3.1.3 角色与职责

角色	职责	类型
产品经理	收集与整理用户诉求, 审核 AI 生成的用户故事	原有
前端开发者	设计功能路由, 定义 TypeScript 接口	原有
AI 需求文档生成器	从路由定义与类型文件中自动生成结构化用户故事	新增 AI 工具角色
AI 一致性验证器	验证路由参数与类型字段的命名、类型与必填/可选一致性	新增 AI 工具角色
技术负责人	确认需求基线	原有

3.1.4 交付物

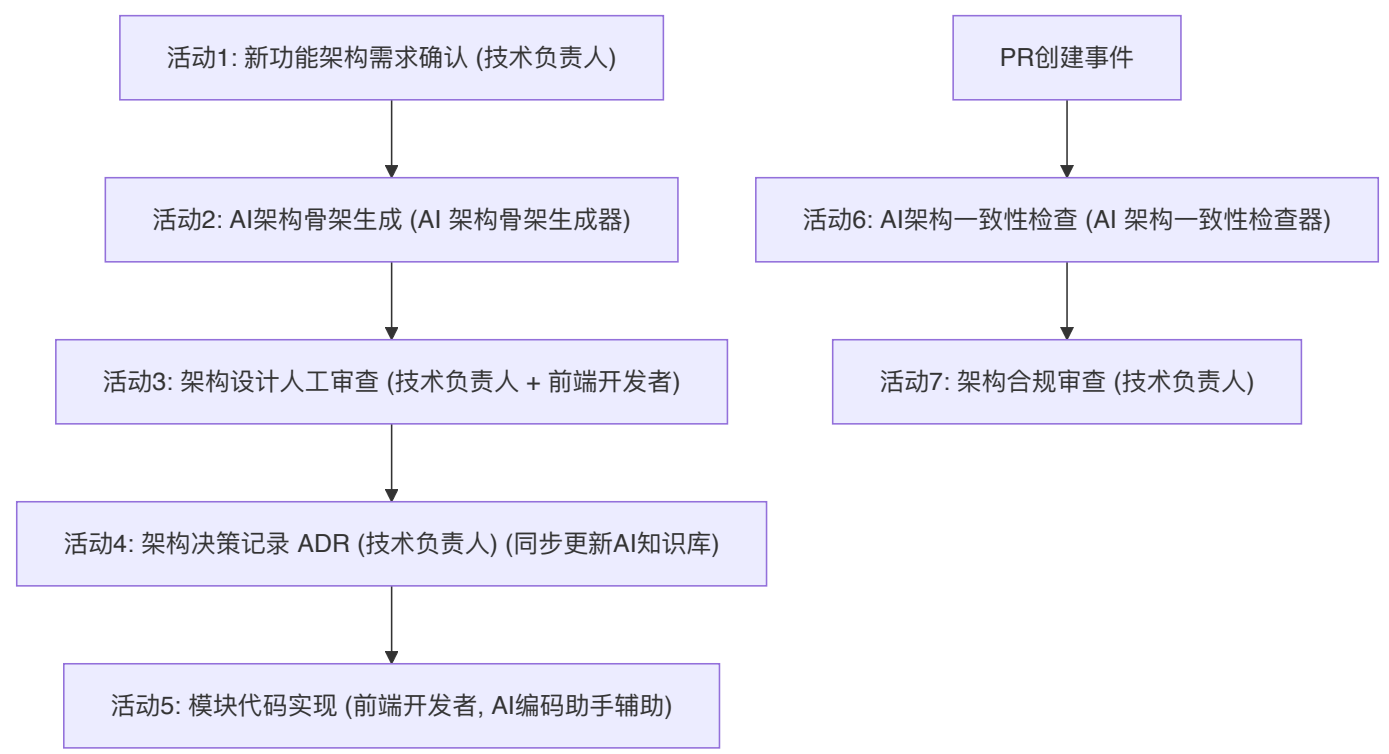
交付物	格式	质量门禁
功能路由定义	<code>src/routes/</code> 路由文件 + <code>routeTree.gen.ts</code>	TypeScript 编译通过
TypeScript 类型定义	<code>src/features/*/types/</code> 接口与类型别名	TypeScript 编译通过
用户故事文档 (AI 辅助生成)	<code>docs/user-stories/</code> Markdown 文件	产品经理审核通过
AI 一致性验证报告	CI 产出物	0 个未解决的不一致项

3.2 体系结构设计过程 (主要过程)

3.2.1 过程概述

体系结构设计过程定义项目顶层结构与模块组织方式. yolink-front 采用领域驱动分层架构: 每个功能领域 (`src/features/<domain>/`) 内部遵循统一的 `api/`、`hooks/`、`components/`、`pages/` 四层分层约定. v2.0 引入 AI 辅助——生成符合分层约定的模块骨架, 在 PR 阶段审查架构一致性.

3.2.2 活动流程



3.2.3 角色与职责

角色	职责	类型
技术负责人	架构决策、ADR 编写、架构一致性最终审查	原有
前端开发者	执行模块实现	原有
AI 架构骨架生成器	基于项目架构模板生成新模块目录结构与骨架代码	新增 AI 工具角色
AI 架构一致性检查器	PR 阶段检测新增模块是否遵循四层分层约定	新增 AI 工具角色

3.2.4 交付物

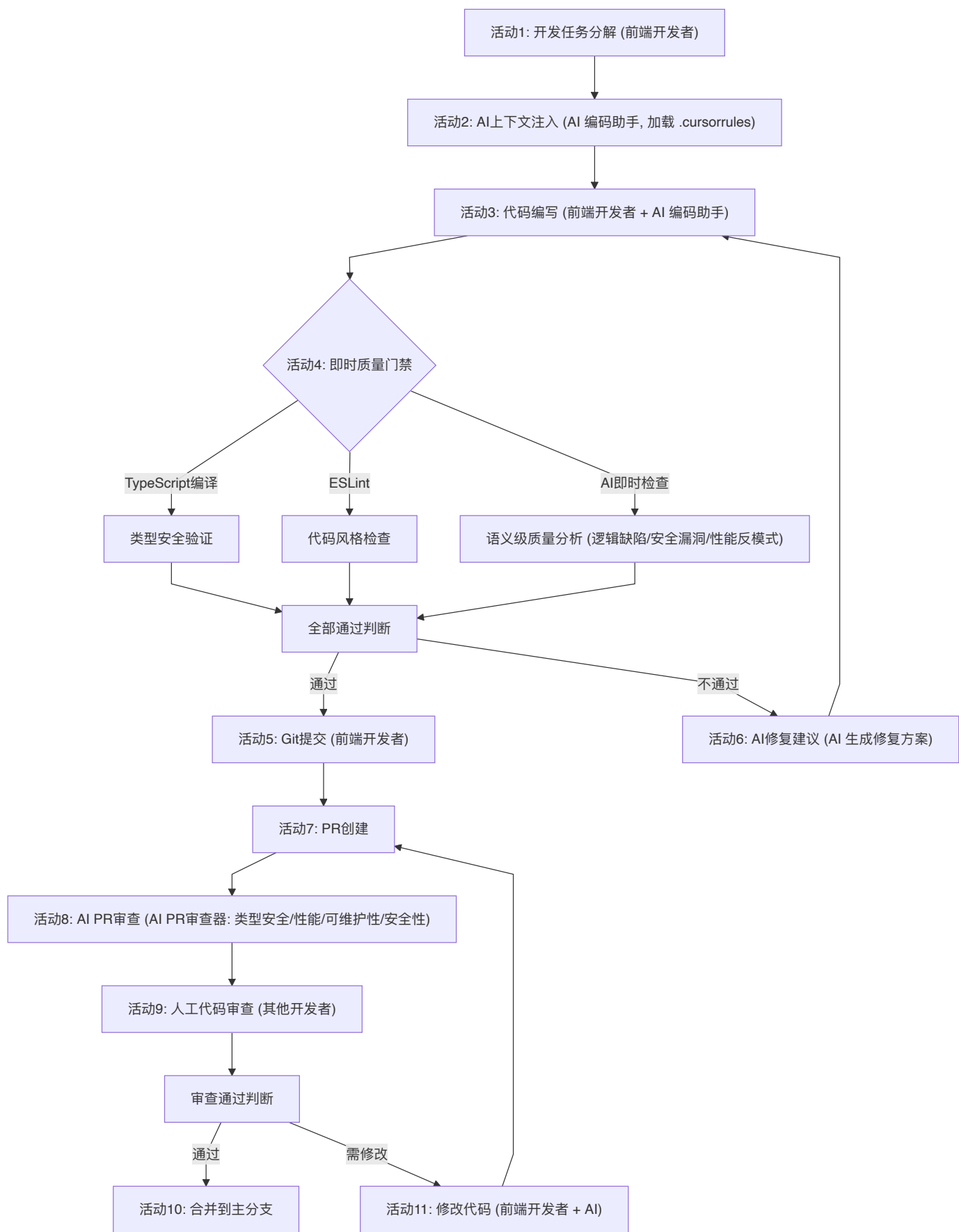
交付物	格式	质量门禁
模块目录结构	<code>src/features/<new-domain>/</code>	遵循 <code>api/hooks/components/pages</code> 结构
架构决策记录 (ADR)	<code>docs/adr/</code> Markdown 文件	技术负责人审批
架构合规检查报告	CI 产出物 (CodeRabbit / Copilot Review)	AI 检查通过 + 人工确认

3.3 软件构造过程 (主要过程)

3.3.1 过程概述

软件构造过程是日常开发活动的核心. 在 TypeScript 严格模式约束下, 每次函数调用与组件 Props 传递都经过编译时类型检验. v2.0 在原有 TypeScript 编译与 ESLint 静态检查基础上, 增加 AI 编码助手 (IDE 级) 与 AI 代码审查 (PR 级) 两道质量防线.

3.3.2 活动流程



3.3.3 角色与职责

角色	职责	类型
前端开发者	代码编写负责人, 对 AI 辅助生成代码进行审查与确认	原有 (角色升级)
AI 编码助手	上下文感知代码补全、多文件编辑、重构建议	新增 AI 工具角色
AI 即时质量检查器	编码阶段检测语义级缺陷	新增 AI 工具角色
AI PR 审查器	PR 阶段多维度自动审查	新增 AI 工具角色
代码审查者	PR 人工审查, 确认 AI 审查结果	原有

3.3.4 交付物

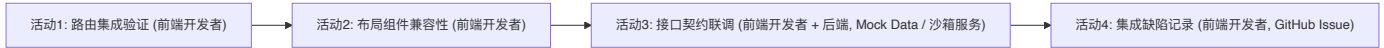
交付物	格式	质量门禁
源代码	TypeScript + React 组件	TypeScript 编译 + ESLint + AI 即时检查通过
PR 描述	Git PR 模板	包含变更说明与测试计划
AI 审查报告	PR Comment (CodeRabbit / Copilot Review)	所有 Critical/High 问题已解决
代码审查记录	PR Review Comments	至少 1 人 approve

3.4 软件集成过程 (主要过程)

3.4.1 过程概述

软件集成过程验证各模块之间的协作正确性. 基于本项目独立前端仓库的工程边界, 集成验证范围限定于前端内部模块集成与接口契约联调——此剪裁决策在 v1.0 中已完成论证. v2.0 维持此剪裁不变, AI 暂不介入该过程. 前端集成验证通过本地开发环境的启动与手动测试完成, 自动化集成测试的价值在当前项目规模下不足以支撑其投入.

3.4.2 活动流程



3.4.3 交付物

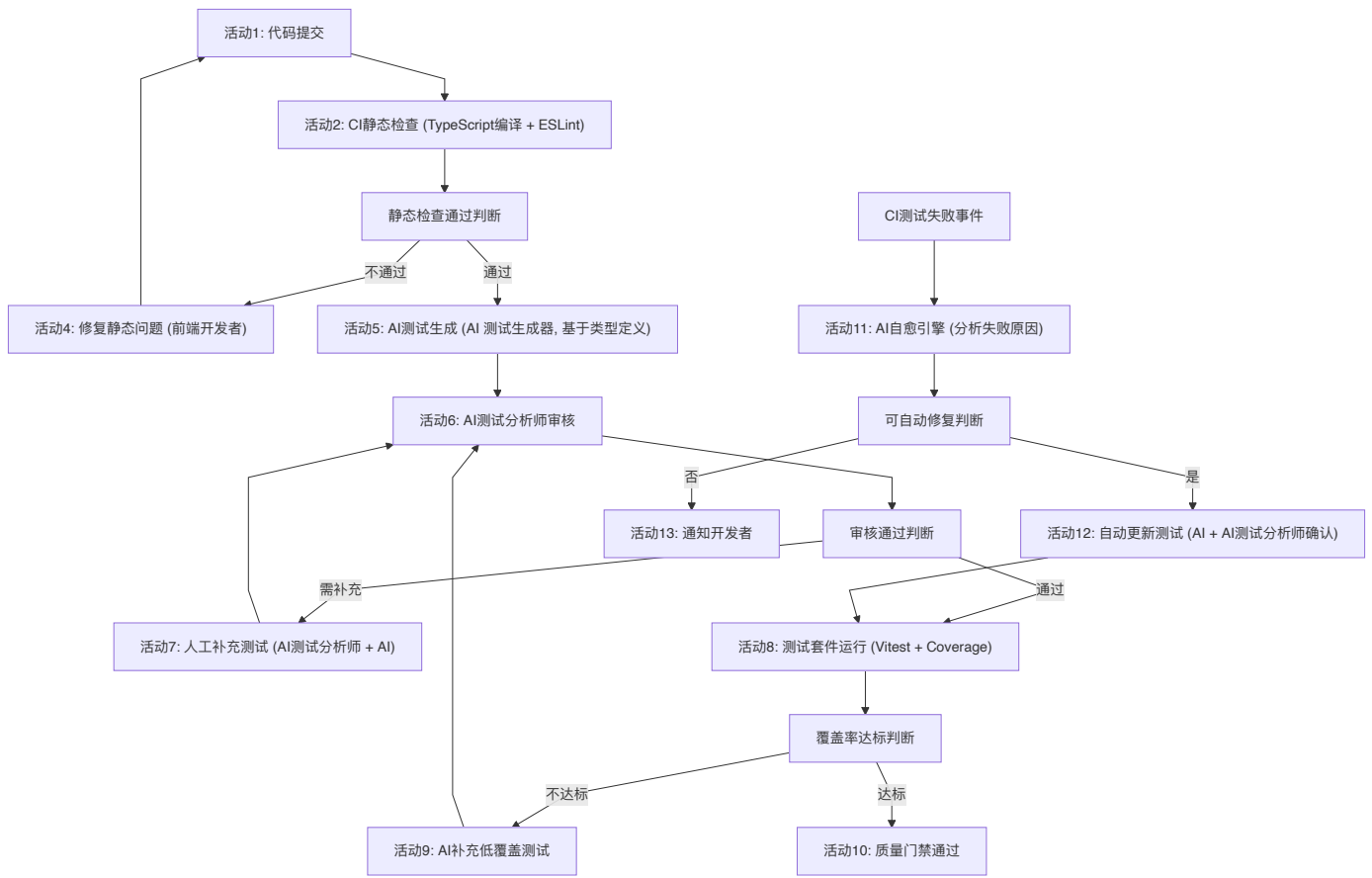
交付物	格式	质量门禁
mockData 配置	<code>src/mock/</code> 目录	覆盖所有 API 端点的正常与异常响应
集成缺陷 Issue	GitHub Issue	已关闭或纳入后续迭代

3.5 质量保证过程 (支持过程)

3.5.1 过程概述

质量保证过程通过多层次验证确保代码符合质量标准. v1.0 仅依赖 TypeScript 编译 + ESLint 静态分析. v2.0 保留这两道防线, 新增 AI 驱动的自动化测试生成与 AI 自愈修复能力, 将质量保证从静态检查升级为静态检查 + 动态验证的双重防护.

3.5.2 活动流程



3.5.3 AI 测试生成 Prompt 模板

AI 训练师为不同类型的被测对象维护差异化的 Prompt 模板:

被测类型	Prompt 关键要素	目标覆盖率
TanStack Query Mutation Hook	Mock API 调用、验证 mutation 状态转换、验证缓存失效、覆盖成功/失败>Loading 状态	>= 85%
TanStack Query Query Hook	Mock 查询函数、验证 loading/success/error 状态、验证 refetch 逻辑	>= 85%
React 组件 (展示型)	渲染测试、Props 边界值测试、事件回调验证、快照测试	>= 80%
React 组件 (交互型)	用户交互模拟、表单验证测试、异步状态变化测试	>= 80%
工具函数	等价类划分、边界值分析、异常输入处理	>= 90%

3.5.4 角色与职责

角色	职责	类型
前端开发者	遵守编码规范, 响应质量门禁失败	原有
AI 测试分析师	审核 AI 生成测试的质量, 制定测试策略, 维护测试预言库	新增
AI 测试生成器	从代码类型定义自动生成 Vitest 测试用例	新增 AI 工具角色
AI 自愈引擎	自动分析测试失败原因并尝试修复	新增 AI 工具角色

3.5.5 交付物

交付物	格式	质量门禁
单元测试套件	<code>*.test.ts</code> / <code>*.test.tsx</code> 文件	TypeScript 编译通过 + 分支覆盖率 >= 80% + AI 测试分析师审核
测试覆盖率报告	CI 产出物 (HTML/JSON)	分支覆盖率 >= 80%, 行覆盖率 >= 85%
AI 测试审核记录	GitHub Issue / PR Comment	AI 测试分析师审核通过
AI 自愈操作日志	CI 日志	所有自动修复均有人工确认标记

3.6 配置管理过程 (支持过程)

配置管理过程以 Git 为核心工具, 主分支 (`main`) 始终维持在可运行状态, 所有功能开发在独立分支上进行, 通过 PR 合并. v2.0 维持此流程不变, 新增 AI 相关配置文件的版本管理.

新增配置项:

配置文件	用途	管理方式
<code>.cursorrules</code>	项目级 AI 编码指令	Git 版本管理, 变更需 AI 训练师审批
<code>.github/copilot-instructions.md</code>	GitHub Copilot 项目指令	Git 版本管理, 变更需 AI 训练师审批
<code>prompts/</code> 目录	AI Prompt 模板库	Git 版本管理
AI 工具版本锁定文件	固定 AI 工具版本与配置	Git 版本管理

3.7 文档管理过程 (支持过程)

3.7.1 过程概述

v1.0 中已确立敏捷文档策略——以 Markdown 笔记、代码注释与 TypeScript 类型定义为主要文档载体. v2.0 将文档管理视角从为人类编写文档扩展为为人类与 AI 共同编写文档. 新增 AI 知识库维护活动, 本质是机器可消费的文档管理.

3.7.2 新增活动



3.7.3 交付物

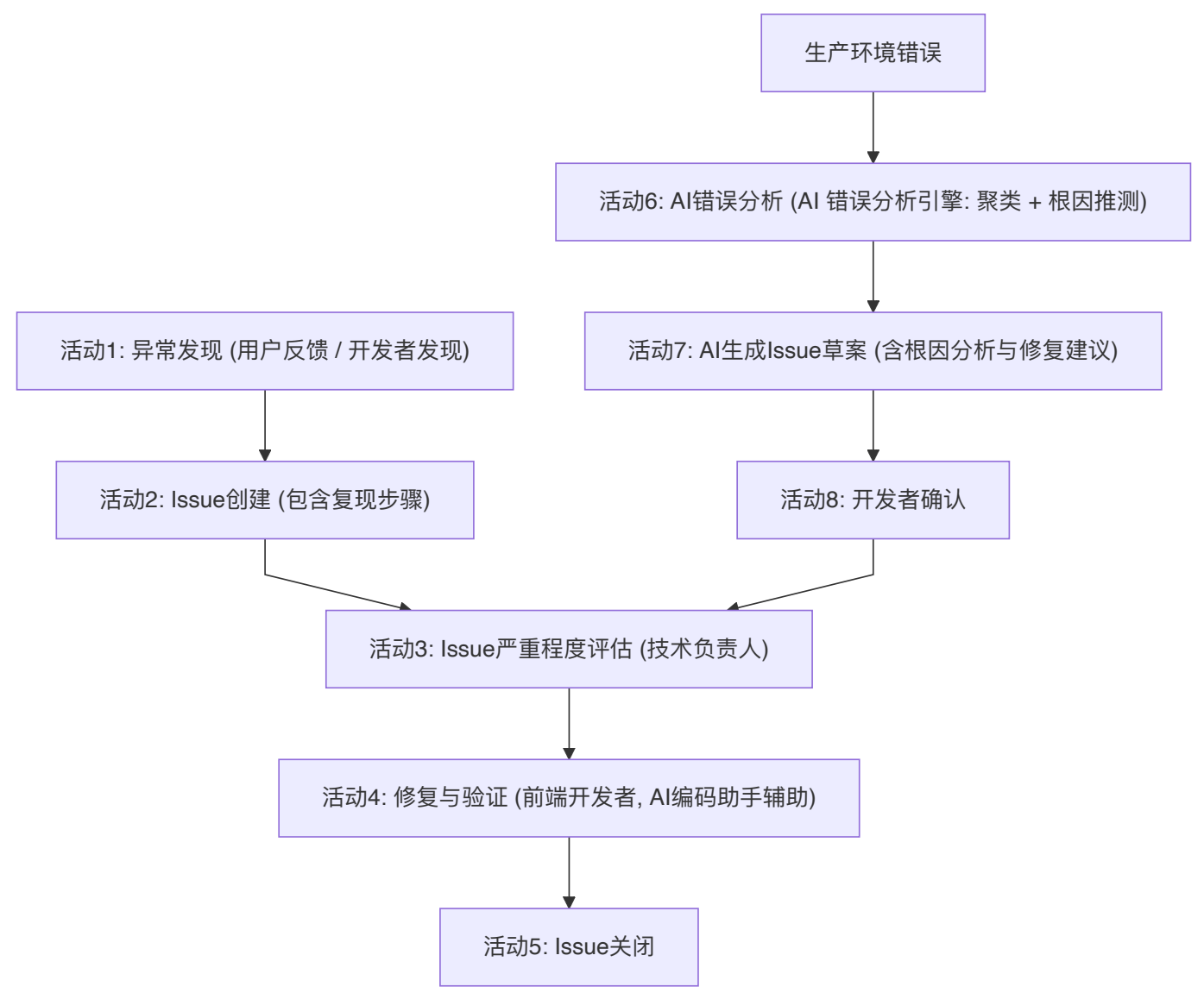
交付物	格式	质量门禁
架构决策记录 (ADR)	docs/adr/	技术负责人审批
过程定义文档	docs/ Markdown 文件	与当前实践一致
AI 指令文件	.cursorrules、.github/copilot-instructions.md	AI 训练师维护
AI Prompt 模板库	prompts/ 目录	AI 训练师维护, 经至少 1 次实际使用验证

3.8 问题解决过程 (支持过程)

3.8.1 过程概述

开发与生产环境中发现的问题通过 GitHub Issue 跟踪. v2.0 在生产环境异常监控链路中集成 AI 驱动的错误聚类与根因分析, 加速从发现问题到定位根因的响应链路.

3.8.2 活动流程



3.8.3 交付物

交付物	格式	质量门禁
GitHub Issue	Issue 描述 + Label + Milestone	包含复现步骤或 AI 根因分析
修复 PR	PR 关联 Issue	回归测试通过
AI 错误分析报告	Issue 评论 (AI 自动生成)	开发者确认

3.9 项目规划过程 (组织过程)

维持 v1.0 不变: 每轮迭代前拆解用户故事, 估算时间成本, 设定 Definition of Done. v2.0 新增 AI 工具成本预算条目, 将 AI API 调用费用纳入迭代成本估算.

3.10 风险管理过程 (组织过程)

v1.0 识别的风险: (1) 第三方库 API 非兼容变更, 应对为 `package.json` 锁定版本; (2) 后端接口不稳定性, 应对为 `axiosInstance.ts` 统一封装. v2.0 新增 5 类 AI 相关风险.

完整风险登记册:

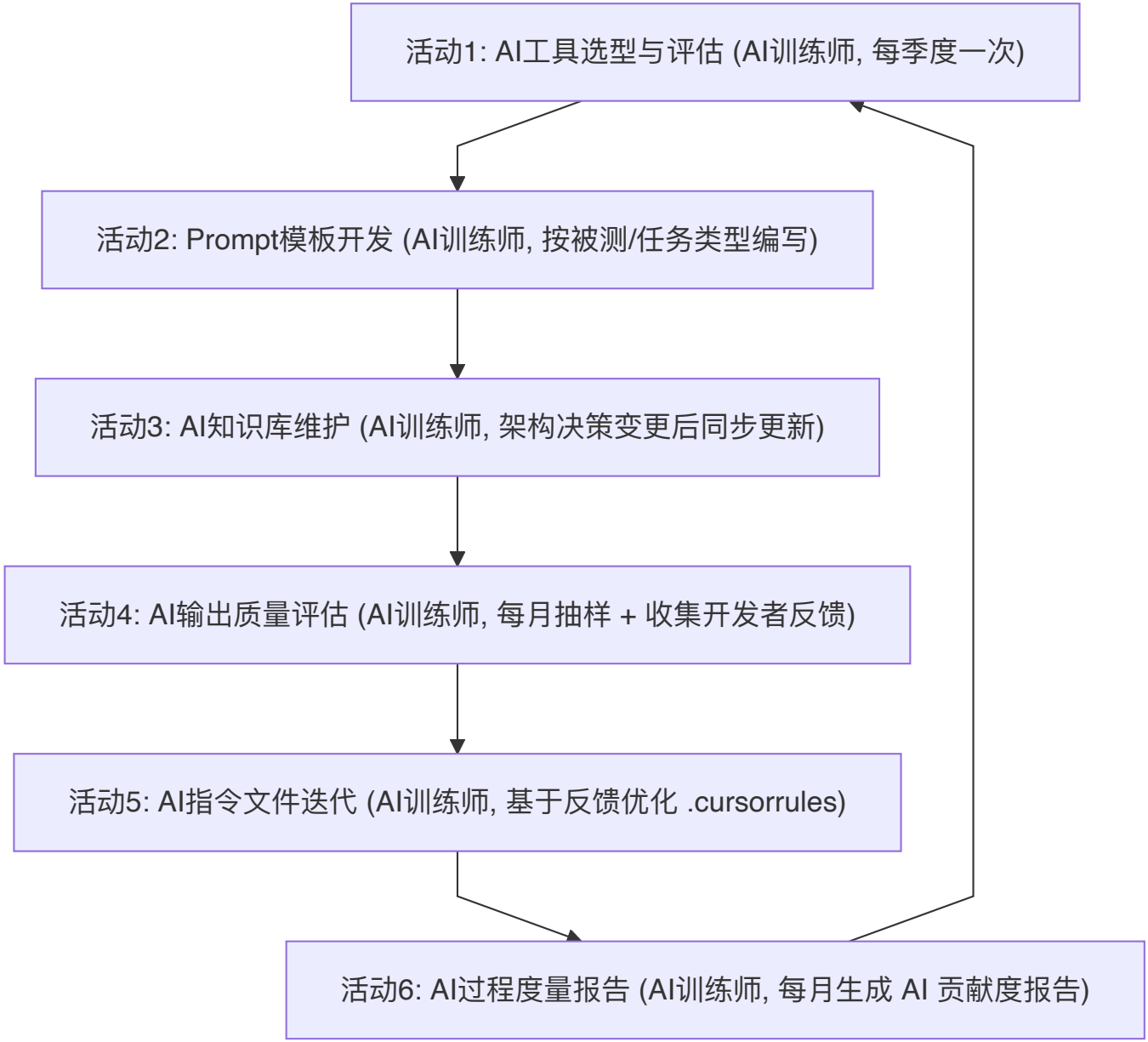
编号	类别	描述	概率	影响	等级	应对策略
R1	技术依赖	第三方库 API 非兼容变更	中	高	高	<code>package.json</code> 锁定版本
R2	技术依赖	后端接口不稳定性	高	中	高	<code>axiosInstance.ts</code> 统一封装
R3	数据安全	AI 工具将源代码发送至云端	中	高	高	企业版数据隔离 + 敏感文件排除
R4	模型偏见	AI 生成代码偏好非项目技术栈	中	中	中	<code>.cursorrules</code> 声明技术栈约束
R5	技术依赖	团队过度依赖 AI 导致核心能力退化	高	高	高	无AI编程日 + 能力监控
R6	输出质量	AI 生成代码存在隐蔽逻辑错误	中	高	高	多层质量门禁 + 双人审查
R7	上下文漂移	AI 知识库与实际架构不同步	高	中	中	每迭代周期审查 + ADR 同步更新
R8	成本膨胀	AI API 调用费用随规模增长失控	中	中	中	月度预算上限 + 费用报告
R9	供应链	AI 工具供应商策略变更	低	高	中	Prompt 标准化封装 + 备选工具评估

3.11 AI 能力管理过程 (组织过程, 新增)

3.11.1 过程概述

AI 能力管理过程是 v2.0 新增的组织过程, 由 AI 训练师主导执行, 负责项目 AI 能力的全生命周期管理——从工具选型、Prompt 工程、知识库维护到输出质量评估.

3.11.2 活动定义



3.11.3 角色与职责

角色	职责	技能要求
AI 训练师	主导 AI 能力管理过程所有活动	LLM Prompt Engineering、React/TypeScript、数据标注与质量评估
技术负责人 (辅助)	审批关键架构知识库变更, 确认 AI 输出质量评估结论	项目架构全局视角
前端开发者 (反馈)	反馈 AI 工具日常使用体验, 参与 AI 输出质量匿名评分	—

3.11.4 交付物

交付物	格式	频率
AI 工具评估报告	Markdown	每季度
Prompt 模板库	<code>prompts/</code> Markdown 文件	持续维护
AI 输出质量评估报告	Markdown	每月
AI 贡献度报告	Markdown (含度量数据)	每月
开发者 AI 满意度问卷结果	电子表格	每月

四、完整角色职责定义

4.1 角色总览

角色	类型	所属过程	核心职责
产品经理	原有人类角色	需求分析	用户诉求收集、需求文档审核
技术负责人	原有人类角色	体系结构设计、风险管理	架构决策、ADR 审批、风险审查
前端开发者	原有人类角色 (升级)	软件构造、集成、问题解决	代码编写主责 + AI 辅助代码确认
代码审查者	原有人类角色	软件构造	PR 人工审查
AI 训练师	新增人类角色	AI 能力管理	AI 工具选型、Prompt 工程、知识库维护、质量评估
AI 测试分析师	新增人类角色	质量保证	AI 测试审核、测试策略制定、自愈修复确认
AI 需求文档生成器	新增 AI 工具角色	需求分析	从路由+类型生成用户故事
AI 一致性验证器	新增 AI 工具角色	需求分析	路由参数与类型字段交叉验证
AI 架构骨架生成器	新增 AI 工具角色	体系结构设计	生成符合分层约定的模块骨架
AI 架构一致性检查器	新增 AI 工具角色	体系结构设计	PR 阶段架构合规自动审查
AI 编码助手	新增 AI 工具角色	软件构造	代码补全、生成、重构建议
AI 即时质量检查器	新增 AI 工具角色	软件构造	编码阶段语义缺陷检测
AI PR 审查器	新增 AI 工具角色	软件构造	PR 多维度自动审查
AI 测试生成器	新增 AI 工具角色	质量保证	基于类型定义生成测试用例
AI 自愈引擎	新增 AI 工具角色	质量保证	测试失败自动分析与修复
AI 错误分析引擎	新增 AI 工具角色	问题解决	生产错误聚类与根因推测

五、交付物清单与质量标准

5.1 完整交付物矩阵

阶段	交付物	格式	负责角色	质量门禁	AI 参与
需求分析	功能路由定义	TypeScript	前端开发者	TypeScript 编译通过	否
需求分析	类型定义	TypeScript	前端开发者	TypeScript 编译通过	否
需求分析	用户故事文档	Markdown	产品经理	审核通过	AI 辅助生成
需求分析	一致性验证报告	CI 产出	AI 一致性验证器	0 个未解决不一致	AI 全自动
体系结构设计	模块目录结构	文件系统	AI 架构骨架生成器	遵循分层约定	AI 辅助生成
体系结构设计	架构决策记录	Markdown	技术负责人	审批通过	否
体系结构设计	架构合规报告	CI 产出	AI 架构一致性检查器	检查通过	AI 全自动
软件构造	源代码	TypeScript + React	前端开发者	编译 + ESLint + AI 检查	AI 辅助 + AI 审查
软件构造	PR 审查报告	PR Comment	AI PR 审查器	Critical/High 全部解决	AI 辅助
软件集成	Mock 数据	TypeScript	前端开发者	覆盖所有 API 端点	否
质量保证	单元测试套件	TypeScript	AI 测试分析师	覆盖率 >= 80% + 审核通过	AI 生成 + 人工审核
质量保证	测试覆盖率报告	CI 产出	Vitest	分支 >= 80%, 行 >= 85%	AI 补充低覆盖
问题解决	GitHub Issue	Issue	前端开发者	含复现步骤/根因分析	AI 辅助生成
AI 能力管理	AI 工具评估报告	Markdown	AI 训练师	每季度	否
AI 能力管理	Prompt 模板库	Markdown	AI 训练师	经实际使用验证	否
AI 能力管理	AI 贡献度报告	Markdown	AI 训练师	每月	否

六、过程度量指标

6.1 传统过程度量指标 (继承 v1.0)

指标	定义	目标
迭代完成率	本迭代完成的 Story Points / 计划完成的 Story Points	>= 85%
缺陷密度	生产缺陷数 / 千行代码	<= 2.0
PR 审查周期	PR 创建到合并的中位时间	<= 24 小时
Issue 关闭周期	Issue 创建到关闭的中位时间	<= 7 天
代码覆盖率	自动化测试分支覆盖率	>= 80%

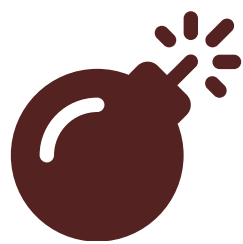
6.2 AI 贡献度指标 (v2.0 新增)

类别	指标名称	定义	目标值	健康区间
效率	AI 编码辅助率	AI 生成代码占总新增代码行数比例	40%-60%	30%-70%
效率	AI 代码接受率	开发者接受 AI 补全建议比例	>= 35%	25%-60%
效率	新模块初始化时间	从零创建到可编译时间	<= 人工的 50%	—
质量	AI 代码缺陷密度	AI 生成代码的生产缺陷数 / AI 代码行数 x 1000	<= 人工代码的 1.2x	<= 1.5x
质量	AI 测试覆盖率增量	AI 生成测试带来的分支覆盖率提升	>= +30pp	—
质量	AI 审查检出率	AI PR 审查发现的真实问题占比	>= 40%	30%-70%
协同	AI 输出人工修改率	人工修改行数 / AI 原始生成行数	20%-40%	10%-50%
协同	AI 自愈成功率	AI 成功修复的失败测试 / 尝试修复数	>= 60%	50%-90%
协同	开发者 AI 满意度	匿名问卷评分 (1-5)	>= 4.0	>= 3.5
成本	AI API 月度费用	月度 AI API 调用总费用	预算内	<= 预算 80% 触发预警

其中 AI 输出人工修改率 是关键健康指标:

- 低于 10%: 可能表明开发者过度信任 AI 输出, 缺少审查
- 高于 50%: 可能表明 Prompt 模板或上下文配置问题, 需 AI 训练师介入

6.3 度量仪表盘



Syntax error in text
mermaid version 11.9.0

七、AI 工具链规范

7.1 推荐工具清单

过程域	工具	主要功能	License/费用	集成方式
软件构造	GitHub Copilot	IDE 内代码补全与生成	Business (\$19/月/人)	VS Code / JetBrains 插件
软件构造	Cursor	AI-first IDE, 多文件编辑	Pro (\$20/月/人)	独立 IDE
软件构造	Claude Code	CLI 级 AI 编程代理, 跨文件重构	API 按量计费	终端 CLI
软件构造 + 设计	CodeRabbit	AI PR 审查 + 架构合规检查	Pro	GitHub App
质量保证	Claude API	测试用例生成	API 按量计费	CI/CD 流水线集成
质量保证	Vitest + Coverage	测试运行与覆盖率统计	开源免费	Node.js 项目依赖
质量保证	Playwright	E2E 测试 (含 AI 测试生成)	开源免费	CI/CD 集成
运维监控	Sentry	前端错误监控 + AI 异常分析	Team	SDK 集成
运维监控	Datadog RUM	真实用户监控 + AI 异常检测	Pro	SDK 集成

7.2 项目级 AI 指令文件规范

`.cursorrules` 核心内容:

```
1 # yolink-front 项目 AI 指令
```

```
2
3  ## 技术栈约束
4  - 框架: React 19 (函数组件 + Hooks)
5  - 语言: TypeScript 5.5+ (strict mode)
6  - 路由: TanStack Router (文件系统路由)
7  - 数据获取: TanStack Query v5
8  - UI 组件库: shadcn/ui
9  - 样式: Tailwind CSS
10 - 构建: Vite
11 - 测试: Vitest + React Testing Library
12
13 ## 项目约定
14 - 不使用 React Router (本项目使用 TanStack Router)
15 - 不使用 Redux、Zustand (服务端状态由 TanStack Query 管理)
16 - 不使用 axios 之外的 HTTP 客户端
17 - 组件导入路径使用 @/ 别名
18 - 新功能模块在 src/features/<feature-name>/ 下创建
19 - 每个 feature 内部遵循 api/hooks/components/pages 分层
20
21 ## 编码规范
22 - 所有 export 函数必须显式标注返回类型
23 - React 组件使用 React.FC<Props> 或显式 Props 参数类型
24 - 禁止使用 any 类型
25 - API 调用的错误必须被捕获并处理
26 - TanStack Query mutation 成功后必须 invalidate 相关查询缓存
```

八、过程合规性声明

8.1 标准符合性

本过程定义文档依据以下标准与文件进行剪裁与扩展:

标准	版本	应用方式
ISO/IEC 12207	2017	过程框架基础, 已剪裁适配独立前端项目
ISO/IEC 33001	2015	过程能力评估框架, 目标 Level 1-2
课程教师指导	2026年春	利益相关方认可剪裁行为与 AI 扩展

8.2 剪裁记录

剪裁项	内容	理由	版本
文档剪裁	不单独编制 SRS/SDD/STP, 以敏捷文档策略替代	代码自解释性 + 类型系统精确性	v1.0 (保持)
集成剪裁	集成验证范围限定于前端内部集成	独立前端仓库的工程边界约束	v1.0 (保持)
AI 扩展	新增 AI 能力管理过程、AI 角色、AI 工具节点	技术进步驱动过程演进	v2.0 (新增)

8.3 过程改进记录

版本	日期	变更	原因
v1.0	2026-03	初始过程定义与剪裁	第一次课后作业
v2.0	2026-06	新增 AI 过程扩展	第二次课后作业——智能化软件过程改进

参考文献

[1] ISO/IEC 12207:2017. Systems and software engineering — Software life cycle processes. ISO, 2017.

[2] ISO/IEC 33001:2015. Information technology — Process assessment — Concepts and terminology. ISO, 2015.

[3] 中国信息通信研究院. 《智能化软件工程技术和应用要求 第3部分: 智能测试能力》. 2024.

[4] GitHub. "GitHub Copilot for Business: Security and Data Protection." GitHub Docs, 2025.

[5] Anthropic. "Claude Code: Agentic Coding Assistant Documentation." Anthropic, 2025.

[6] Gtest 全球软件测试技术峰会. "AI 驱动测试自动化: 从理论到实践." Gtest 2025 会议资料.

[7] 《AI4SE 行业现状调查报告》. AI4SE 工作组, 2024.

[8] Chen, M., et al. "Evaluating Large Language Models for Software Testing." Proceedings of ICSE 2025, IEEE, 2025.

[9] Zhou, X., et al. "AI-Native Test Automation: A New Paradigm." IEEE Transactions on Software Engineering, vol. 51, no. 4, 2025.

[10] Liu, H., et al. "Generative AI in Requirements Engineering: Opportunities and Challenges." ACM Computing Surveys, vol. 57, no. 3, 2025.

[11] GitHub Next. "The State of AI in Software Development: 2025 Annual Report." GitHub, 2025.

[12] Stack Overflow. "2025 Developer Survey: AI Tools in the Development Workflow." 2025.